UNIT – 3

Understanding Supervised Learning

602
Data Analytics using Python

Contents

3.1 Overview of Supervised Learning 1
3.1.1 Concepts of Supervised Learning 1
3.1.2 Difference between Classification and Regression 3
Examples of Classification and Regression 3
3.2 Basic Terminologies 8
3.2.1 Dataset, Features, Labels 8
3.2.2 Training Data, Test Data, Validation Data 9
3.3.3 Overfitting and Underfitting 10
3.3 Loss Functions 11
3.3.1 Mean Squared Error (MSE) 11
3.3.2 Definition of MSE 11
3.3.3 Mean Absolute Error (MAE) 12

3.1 Overview of Supervised Learning

- **Supervised Learning** is a type of machine learning where the model is trained using **labeled data**.
- It is defined by its use of **labeled datasets** to train algorithms to classify data or predict outcomes accurately.
- Think of it as a student learning under the supervision of a teacher who already knows the answers.
- Each training example consists of:
    - **Input features**
    - **Correct output (label)**
- The model learns a mapping between inputs and outputs and uses this learned relationship to make predictions on new, unseen data.

General Workflow of Supervised Learning

- Collect labeled dataset
- Split data into training and testing sets
- Train the model using training data
- Evaluate model performance using test data
- Use the trained model for prediction

3.1.1 Concepts of Supervised Learning

- In supervised learning, the model is provided with a set of input-output pairs.
- The goal is to learn a mapping function (f) that maps the input variables (X) to the output variable (Y).

$$Y = f(X)$$

1. Labeled Dataset

In supervised learning, every data point has a known output.

Example:

- Email dataset
    - Input: Email text

- Label: Spam or Not Spam

2. Features and Labels

- **Features** are input variables used for prediction. The independent variables or attributes used as input (e.g., square footage of a house, age of a car).
- **Labels/Targets** are the target outputs. The dependent variable or label we want to predict (e.g., price of a house, type of fruit).

Example (Student Dataset):

- Features: Study hours, attendance
- Label/Targets: Pass or Fail

3. Training Phase

- The model learns patterns from the training data by comparing predicted outputs with actual labels and minimizing error using a loss function or error function.

4. Testing Phase

- After training, the model is tested on unseen data to check how well it generalizes.

5. Feedback Mechanism

- Errors made by the model are used to adjust model parameters and improve accuracy.

Types of Supervised Learning

- Classification
- Regression

Real-Life Examples of Supervised Learning

- Email spam detection
- House price prediction
- Sentiment analysis
- Medical diagnosis

3.1.2 Difference between Classification and Regression

Classification

- Classification is a supervised learning technique where the output variable is **categorical**.
- **Example Outputs:** Yes / No, Spam / Not Spam, Pass / Fail

Regression

- Regression is a supervised learning technique where the output variable is **continuous** or numerical.
- **Example Outputs:** House price, Temperature, Salary

Point of Difference	Classification	Regression
Type of output	Categorical	Continuous
Nature of prediction	Class or category	Numerical value
Example output	Spam or Not Spam	₹45,000
Common algorithms	Logistic Regression, Decision Tree, KNN	Linear Regression, Ridge, Lasso
Evaluation metrics	Accuracy, Precision, Recall	MSE, MAE, R^2
Use case	Email filtering	Price prediction
Graphical representation	Decision boundaries	Best-fit line or curve

Examples of Classification and Regression

Example-1: (Classification)

```
#Import libraries
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
import pandas as pd

#Load the iris data set
iris = datasets.load_iris()
```

```
#checking features
#Input features (feature_names)
print("Input columns --->",iris.feature_names)

#Output features (target_names)
print("Output columns --->",iris.target_names)

#checking input data values (data)
print("Input values ---> ",iris.data)

#checking output data values (target)
print("Input values ---> ",iris.target)

#assigning values to x & y
x = iris.data
y = iris.target

#checking the size/shape on input and output values
print(x.shape)
#checking the size/shape on input and output values
print(y.shape)

#Build Classification Model using Random Forest and fitting it
clf = RandomForestClassifier()
clf.fit(x,y)
#Feature Importance
print(clf.feature_importances_)

#Make Prediction (predict or predict_proba)
print(x[0])
# [[1,2,3],[4,5,6],[7,8,9]]
clf.predict(x[[10]])

#predicting class names instead of numbers --->
fit(x.data,y.target_names[iris.target])
clf.fit(x,iris.target_names[iris.target])
print(clf.predict_proba([[8.8,10.2,25.6,21.5]]))
print(clf.predict([[8.8,10.2,25.6,21.5]]))
```

```
s_len = float(input("Enter sepal length (cm):"))
s_wid = float(input("Enter sepal width (cm):"))
p_len = float(input("Enter petal length (cm):"))
p_wid = float(input("Enter petal width (cm):"))
print(clf.predict([[s_len,s_wid,p_len,p_wid]]))
```

Example-2: (Classification with splitting the dataset)

```
from sklearn.model_selection import train_test_split

#train_test_split(x,y,test_size,random_state)
x_train, x_test, y_train, y_test =
train_test_split(x,y,test_size=0.2,random_state=42)

clf = RandomForestClassifier(random_state=42)
clf.fit(x_train,y_train)
print(x_train[0])
print(x_test[10])
print(clf.predict(x_test[[10]]))
```

Example-3: (Classification using DecisionTreeClassifier)

```
import pandas as pd
from sklearn import datasets
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
from sklearn import tree

# 1. Load the Iris dataset (built-in to scikit-learn for easy example)
iris = datasets.load_iris()
X = iris.data  # Features
y = iris.target  # Target variable (species of iris flower)

# 2. Split the data into training and testing sets (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=1)
```

```
# 3. Create a Decision Tree classifier object
# We use the Gini index as the criterion for the split quality
clf = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=1)

# 4. Train the Decision Tree classifier model
clf.fit(X_train, y_train)

# 5. Make predictions on the test set
y_pred = clf.predict(X_test)

# 6. Evaluate the model's accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")

# 7. Visualize the trained Decision Tree (optional, but very helpful)
plt.figure(figsize=(12, 8))
tree.plot_tree(clf,
               feature_names=iris.feature_names,
               class_names=iris.target_names,
               filled=True,
               rounded=True,
               fontsize=10)
plt.show()
```

Example-4: (Regression using RandomForestRegressor)

```
# Import libraries
from sklearn import datasets
from sklearn.ensemble import RandomForestRegressor

# Load regression dataset
diabetes = datasets.load_diabetes()

# checking features
print("Input columns --->", diabetes.feature_names)

# checking output
print("Output column ---> Disease Progression")
```

```
# assigning values to x & y
x = diabetes.data
df = pd.DataFrame(x)
print(df.head(5))
y = diabetes.target
df_y = pd.DataFrame(y)
print(df_y.head(5))

# checking shapes
print("Input shape --->", x.shape)
print("Output shape --->", y.shape)

# Build Regression Model
reg = RandomForestRegressor(random_state=42)
reg.fit(x, y)

# Feature importance
print("Feature importance --->", reg.feature_importances_)

# Prediction
print("First input row --->", x[0])
print("Predicted value --->", reg.predict(x[[0]]))

# User input prediction
bmi = float(input("Enter BMI value: "))
bp = float(input("Enter BP value: "))
s1 = float(input("Enter S1: (Total serum cholesterol) "))
s2 = float(input("Enter S2: (Low-density lipoproteins) "))
s3 = float(input("Enter S3: (High-density lipoproteins) "))
s4 = float(input("Enter S4: (Total cholesterol / HDL) "))
s5 = float(input("Enter S5: (Log of serum triglycerides)"))
s6 = float(input("Enter S6: (Blood sugar level) "))
age = float(input("Enter Age: "))
sex = float(input("Enter Sex (0/1): "))

print("Predicted disease progression --->",
    reg.predict([[age, sex, bmi, bp, s1, s2, s3, s4, s5, s6]]))
```

Example-4: (Regression using LinearRegression)

```
# Regression Analysis using Linear Regression
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Step 1: Load the CSV file
df = pd.read_csv("StudentsPerformance.csv")

# Step 2: Separate features and target
X = df[['writing score']] # Independent variable (2D)
y = df['math score'] # Dependent variable (1D)

model = LinearRegression()
model.fit(X, y)

y_pred = model.predict(X)
print(y_pred)

#X_for_plotting = np.linspace(min(X), max(X), 100)
plt.scatter(X,y,color='blue')
plt.plot(X, y_pred, color='red', label='Regression Line')
plt.show()
```

3.2 Basic Terminologies

3.2.1 Dataset, Features, Labels

Dataset

A dataset is a structured collection of data used to train, validate, and test a machine learning model. It is usually organized in tabular form where each row represents one data instance and each column represents an attribute.

Example:

- A dataset containing information about students with columns such as marks, attendance, and final result.

Features

Features are the input variables or attributes used by the model to make predictions. They describe the characteristics of each data instance.

Example:

- In a student performance dataset:
    - Marks
    - Attendance percentage
    - Study hours

These are features.

Labels

Labels are the output variables or target values that the model tries to predict. They represent the correct answer for each data instance.

Example:

- Pass or Fail
- Grade A, B, or C

These are labels.

3.2.2 Training Data, Test Data, Validation Data

Training Data

Training data is the portion of the dataset used to teach the model. The model learns patterns and relationships between features and labels from this data.

Example:

- If a dataset has 1000 records, around 70 percent may be used as training data.

Validation Data

Validation data is used to tune model parameters and make decisions such as model selection or stopping training. It helps in improving model performance without using test data.

Purpose:

- Hyperparameter tuning
- Preventing overfitting

Test Data

Test data is used only after the model has been fully trained. It evaluates how well the model performs on unseen data.

Important Point:

- The test dataset must not be used during training or validation.

Typical Data Split Example

- Training data: 70 percent
- Validation data: 15 percent
- Test data: 15 percent

3.3.3 Overfitting and Underfitting

Overfitting

Overfitting occurs when a model learns the training data too well, including noise and unnecessary details. As a result, it performs very well on training data but poorly on new or unseen data.

Characteristics:

- High training accuracy
- Low testing accuracy

Causes:

- Too complex model
- Too many features
- Insufficient training data

Example:

A student memorizing answers instead of understanding concepts.

Underfitting

Underfitting occurs when a model is too simple to capture the underlying patterns in the data. It performs poorly on both training and test data.

Characteristics:

- Low training accuracy
- Low testing accuracy

Causes:

- Model is too simple
- Important features are missing

- Insufficient training time

Example:

A student who studies too little and fails to understand the subject.

3.3 Loss Functions

- A **loss function** measures how far a model's predictions are from the actual values. It helps in **evaluating model performance** and **guides the learning process** by minimizing the error.
- Loss functions are especially important in **regression problems**, where the output is a continuous value.

3.3.1 Mean Squared Error (MSE)

- **Mean Squared Error (MSE)** calculates the average of the squared differences between the actual values and the predicted values.
- It penalizes larger errors more heavily due to squaring.

3.3.2 Definition of MSE

- y = actual value
- $\hat{y}$ = predicted value
- n = number of data points

Formula

$$\text{MSE} = \frac{\Sigma\,(y - \hat{y})^2}{n}$$

3.3.2.1 Computing MSE and Its Properties

Example: Computing MSE

Actual (y)	Predicted ($\hat{y}$)	Error ($y - \hat{y}$)	Squared Error
10	8	2	4
15	14	1	1
20	18	2	4

MSE = (4 + 1 + 4) / 3 = **3**

Properties of MSE

- Always **non-negative**
- Penalizes **large errors more strongly**

- Differentiable, making it suitable for optimization
- Sensitive to **outliers**

3.3.3 Mean Absolute Error (MAE)

- **Mean Absolute Error (MAE)** calculates the average of the absolute differences between actual and predicted values.
- It treats all errors equally.

3.3.3.1 Definition of MAE

Formula

$$\text{MAE} = \frac{\Sigma\,|y - \hat{y}|}{n}$$

3.3.3.2 Computing MAE and Its Properties

Example: Computing MAE

Actual (y)	Predicted (ŷ)	Absolute Error |y- ŷ|
10	8	2
15	14	1
18	20	2

MAE = (2 + 1 + 2) / 3 = **1.67**

Properties of MAE

- Always **non-negative**
- Less sensitive to outliers compared to MSE
- Easy to interpret
- Not differentiable at zero (less smooth)

Difference Between MSE and MAE

Aspect	MSE	MAE
Error treatment	Squares errors	Absolute errors
Sensitivity to outliers	High	Low
Optimization	Smooth	Less smooth
Interpretation	Less intuitive	More intuitive

3.3.3.3 Understanding Regression and R^2

What is Regression?

- **Regression** is a supervised learning technique used to predict **continuous values** such as:
    - House prices
    - Salary
    - Temperature

R^2 Score (Coefficient of Determination)

- **R^2** measures how well the regression model explains the variability of the output data.
- **R^2** measures the proportion of variance in dependent variable explained by the independent variable.

Formula

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Where:

- SS_{res} = sum of squared residuals (squared error of the model) **(actual values - predicted values)^2**
- SS_{tot} = total sum of squares (the sum of squared differences between the actual data and the mean of the data) **(actual values – mean of the data)**

Interpretation of R^2

R^2 Value	Meaning
1	Perfect prediction (perfect fit)
0	No improvement over mean
Negative	Poor model

Example of R^2

- Actual values vary widely
- Model predictions closely match actual values

Then R^2 will be **close to 1**, indicating a good model.

Real-Life Example (House Price Prediction)

- **MSE** shows how large prediction errors are, penalizing large mistakes

- **MAE** shows average error in currency units
- **R²** shows how well the model explains house price variation

Example-1: Calculating LOSS FUNCTION using in-built function:

```
# Import required libraries
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Actual house prices
y_actual = [50, 60, 70, 80]

# Predicted house prices
y_predicted = [48, 62, 68, 85]

# Calculate Mean Squared Error
mse = mean_squared_error(y_actual,y_predicted)
print(mse)

# Calculate Mean Absolute Error
mae = mean_absolute_error(y_actual,y_predicted)
print(mae)

# Calculate R2 score
r2 = r2_score(y_actual,y_predicted)
print(r2)
```

Example-2: Calculating LOSS FUNCTION using in-built function:

```
import numpy as np
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Sample Data: Actual values (y) and Predicted values (y_hat)
y_true = np.array([3.0, -0.5, 2.0, 7.0])
y_pred = np.array([2.5, 0.0, 2.1, 7.8])

# Compute MSE
mse_val = mean_squared_error(y_true, y_pred)

# Compute MAE
```

```
mae_val = mean_absolute_error(y_true, y_pred)

# Compute R2 Score
r2_val = r2_score(y_true, y_pred)

print(f"Mean Squared Error (MSE): {mse_val:.4f}")
print(f"Mean Absolute Error (MAE): {mae_val:.4f}")
print(f"R-squared (R2): {r2_val:.4f}")
```

Example-3: Calculating LOSS FUNCTION using manual function:

```
# Manual MSE: Mean of (errors squared)
manual_mse = np.mean((y_true - y_pred)**2)

# Manual MAE: Mean of (absolute errors)
manual_mae = np.mean(np.abs(y_true - y_pred))

# Manual R2: 1 - (SS_res / SS_tot)
ss_res = np.sum((y_true - y_pred)**2)
ss_tot = np.sum((y_true - np.mean(y_true))**2)
manual_r2 = 1 - (ss_res / ss_tot)

print(f"Manual MSE: {manual_mse:.4f}")
print(f"Manual MAE: {manual_mae:.4f}")
print(f"Manual R2:  {manual_r2:.4f}")
```

Prepared by:

Dr. Pooja Negi

Ms. Aarti Agarwal